

Lab: Immersion

“...We learn by doing.”
-- Aristotle

In pairs, build your own Battleship game. Create a clean project, set up your own code repository, and be mindful of writing all code **Test-Driven**. (Be warned: You may be visited by the “Untested-Code-Eating Shark” who looks a lot like me.)

The basics:

- Two players.
- Two 10 x 10 grids.
- Each player's ships are arranged horizontally or vertically (and secretly). A player's ships cannot contact each other.
- Ships vary in size.
- Players call out salvo coordinates to try to hit the other player's ships.
- When all squares of a ship are hit, the ship is sunk.
- The player who sinks all the other player's ships wins.

	A	B	C	D	E	F	G	H	I	L
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

- Fleet composition (from Milton Bradley and Wikipedia)

Type of ship	Size	Type of ship	Size
aircraft carrier	5	aircraft carrier	5
battleship	4	battleship	4
destroyer	3	cruiser	3
submarine	3	submarine	3
patrol boat	2	destroyer	2

Put the software to work:

- Have the game randomly place the ships on the boards for the players. A player's ships cannot overlap or be adjacent to each other.
- Have the game be one of the players. I assure you that this does **not** require an AI decision tree with Alpha-Beta Pruning. Walk through your own strategies.

A suggested approach:

1. Start with the domain model (no UI, no persistence). Do a little whiteboard UML or CRC cards, as a pair, or ask another pair to join you for this part. Set a timer for 10-15 minutes. Just enough to identify some clear behaviors. Think about what behaviors you need, rather than what data-structures you need.
2. Get the basics working. Test behaviors, and let objects and data structures emerge to support those behaviors.
3. *Put the software to work.* This is where your TDD skills will really be sharpened.
4. If there's time: Create a console app or simple JavaScript UI and play a few rounds.
5. Then, be the customer, and add fun variations.

Be sure to manage your own cadence and breaks. Perhaps set up 50-minute time-boxes using a smartphone timer.